

MANUAL DEL INTEGRANTE 3 — BACKEND 2 (PERSISTENCIA Y SERVICIOS)

Proyecto

Sistema de diseño y optimización de redes de fibra óptica utilizando algoritmos voraces, JavaFX y PostGIS.

1. OBJETIVO DEL MÓDULO

Este módulo será responsable de:

- Conectar Java con PostgreSQL.
 - Gestionar PostGIS.
 - Guardar calles.
 - Guardar nodos.
 - Guardar conexiones.
 - Realizar consultas espaciales.
 - Validar ubicaciones.
 - Construir grafos para Backend 1.
 - Comunicar Frontend con Base de Datos.
 - Comunicar Frontend con Backend 1.
-

NO ES RESPONSABILIDAD DE ESTE MÓDULO

No deberá trabajar con:

- JavaFX
 - FXML
 - Leaflet
 - Diseño visual
 - Prim
 - Kruskal
 - Dijkstra
 - Optimización
-

2. ESTRUCTURA DEL PROYECTO

```
backend2/  
|
```

```
├── config/
│   └── DatabaseConnection.java
├── dto/
│   ├── NodoDTO.java
│   ├── CalleDTO.java
│   ├── ConexionDTO.java
│   └── ResultadoDTO.java
├── repositories/
│   ├── NodoRepository.java
│   ├── CalleRepository.java
│   └── ConexionRepository.java
├── services/
│   ├── NodoService.java
│   ├── CalleService.java
│   ├── ConexionService.java
│   ├── SpatialValidationService.java
│   └── GrafoBuilderService.java
├── mapper/
│   └── NodoMapper.java
└── exceptions/
    └── SpatialException.java
```

3. ARQUITECTURA

```
Frontend
↓
Service
↓
Repository
↓
PostgreSQL/PostGIS
```

IMPORTANTE

Frontend nunca debe acceder directamente a:

```
NodoRepository  
CalleRepository  
ConexionRepository
```

4. CLASE DatabaseConnection

Ubicación:

```
config/DatabaseConnection.java
```

RESPONSABILIDAD

Crear conexión JDBC.

ATRIBUTOS

```
private static final String URL;  
  
private static final String USER;  
  
private static final String PASSWORD;
```

MÉTODO

getConnection()

```
public static Connection  
getConnection()
```

Retorna:

```
Connection
```

COMENTARIO

Toda consulta SQL deberá utilizar esta clase.

5. DTO NodoDTO

Ubicación:

dto/NodoDTO.java

RESPONSABILIDAD

Transportar información entre capas.

ATRIBUTOS

```
private int id;

private String nombre;

private String tipo;

private String estado;

private int capacidadMax;

private int clientesActuales;

private double latitud;

private double longitud;
```

COMENTARIO

No contiene lógica.

Solo datos.

6. DTO CalleDTO

Ubicación:

```
dto/CalleDTO.java
```

ATRIBUTOS

```
private int id;  
  
private String nombre;  
  
private String tipoVia;  
  
private double longitud;
```

7. DTO ConexionDTO

Ubicación:

```
dto/ConexionDTO.java
```

ATRIBUTOS

```
private int id;  
  
private int origenId;  
  
private int destinoId;  
  
private double distancia;
```

8. CLASE NodoRepository

Ubicación:

repositories/NodoRepository.java

RESPONSABILIDAD

Acceder a tabla:

nodos

MÉTODO

guardar()

```
public NodoDTO guardar(  
    NodoDTO nodo  
)
```

Retorna:

NodoDTO

FUNCIÓN

Insertar nodo en PostgreSQL.

MÉTODO

buscarPorId()

```
public NodoDTO buscarPorId(  
    int id  
)
```

Retorna:

NodoDTO

MÉTODO

listarTodos()

```
public List<NodoDTO>  
listarTodos()
```

Retorna:

List<NodoDTO>

MÉTODO

eliminar()

```
public void eliminar(  
    int id  
)
```

Retorna:

void

COMENTARIO

Cuando se elimina un cliente o poste.

9. CLASE CalleRepository

Ubicación:

repositories/CalleRepository.java

RESPONSABILIDAD

Acceder a tabla:

calles

MÉTODO

listarCalles()

```
public List<CalleDTO>  
listarCalles()
```

Retorna:

List<CalleDTO>

COMENTARIO

Método utilizado por el mapa.

10. CLASE ConexionRepository

Ubicación:

repositories/ConexionRepository.java

RESPONSABILIDAD

Gestionar tabla:

conexiones

MÉTODO

guardar()

```
public ConexionDTO guardar(  
    ConexionDTO conexion  
)
```

Retorna:

ConexionDTO

MÉTODO

listarTodas()

```
public List<ConexionDTO>  
listarTodas()
```

Retorna:

List<ConexionDTO>

11. CLASE SpatialValidationService

Ubicación:

services/SpatialValidationService.java

RESPONSABILIDAD

Validar ubicaciones utilizando PostGIS.

IMPORTANTE

Solo valida.

No guarda información.

MÉTODO

validarPoste()

```
public boolean validarPoste(  
    double latitud,  
    double longitud  
)
```

Retorna:

```
boolean
```

FUNCIÓN

Verificar que exista una calle cercana.

COMENTARIO

Utilizar consulta PostGIS.

ESCENARIO

Si retorna:

```
false
```

No se debe guardar el poste.

MÉTODO

distanciaACalle()

```
public double distanciaACalle(  
    double latitud,  
    double longitud  
)
```

Retorna:

```
double
```

FUNCIÓN

Obtener distancia mínima a una calle.

12. CLASE NodoService

Ubicación:

```
services/NodoService.java
```

RESPONSABILIDAD

Gestionar nodos.

IMPORTANTE

Frontend debe utilizar esta clase.

MÉTODO

crearNodo()

```
public NodoDTO crearNodo(  
    NodoDTO nodo  
)
```

Retorna:

NodoDTO

FLUJO

```
NodoDTO  
↓  
Validación espacial  
↓  
Guardar  
↓  
Retornar nodo
```

COMENTARIO

Si el nodo es un poste:

SpatialValidationService

debe validar ubicación.

MÉTODO

obtenerNodo()

```
public NodoDTO obtenerNodo(  
    int id  
)
```

Retorna:

```
NodoDTO
```

MÉTODO

listarNodos()

```
public List<NodoDTO>  
listarNodos()
```

Retorna:

```
List<NodoDTO>
```

MÉTODO

eliminarNodo()

```
public void eliminarNodo(  
    int id  
)
```

Retorna:

```
void
```

13. CLASE CalleService

Ubicación:

```
services/CalleService.java
```

RESPONSABILIDAD

Servir calles al Frontend.

MÉTODO

obtenerCalles()

```
public List<CalleDTO>  
obtenerCalles()
```

Retorna:

```
List<CalleDTO>
```

COMENTARIO

MapController utilizará este método.

14. CLASE ConexionService

Ubicación:

```
services/ConexionService.java
```

RESPONSABILIDAD

Gestionar conexiones persistentes.

MÉTODO

guardarConexion()

```
public ConexionDTO guardarConexion(  
    ConexionDTO conexion  
)
```

Retorna:

```
ConexionDTO
```

MÉTODO

listarConexiones()

```
public List<ConexionDTO>  
listarConexiones()
```

Retorna:

```
List<ConexionDTO>
```

15. CLASE GrafoBuilderService

Ubicación:

```
services/GrafoBuilderService.java
```

RESPONSABILIDAD

Construir objetos Grafo para Backend 1.

COMENTARIO

Esta es la clase puente entre Backend 2 y Backend 1.

MÉTODO

construirGrafo()

```
public Grafo construirGrafo()
```

Retorna:

```
Grafo
```

FLUJO

```
Leer nodos
↓
Leer conexiones
↓
Construir grafo
↓
Enviar a Backend 1
```

16. CLASE NodoMapper

Ubicación:

```
mapper/NodoMapper.java
```

RESPONSABILIDAD

Convertir:

NodoDTO

a

Nodo

y viceversa.

MÉTODO

toModel()

```
public Nodo toModel(  
    NodoDTO dto  
)
```

Retorna:

Nodo

MÉTODO

toDTO()

```
public NodoDTO toDTO(  
    Nodo nodo  
)
```

Retorna:

NodoDTO

17. COMUNICACIÓN CON BACKEND 1

Clase utilizada:

```
GrafoBuilderService
```

FLUJO

```
Base de Datos  
↓  
DTO  
↓  
NodoMapper  
↓  
Grafo  
↓  
Backend 1
```

MÉTODO PRINCIPAL

```
public Grafo construirGrafo()
```

18. COMUNICACIÓN CON FRONTEND

Frontend nunca debe acceder a Repository.

CREAR NODO

Frontend:

```
nodoService.crearNodo(dto);
```

OBTENER CALLES

Frontend:

```
calleService.obtenerCalles();
```

LISTAR NODOS

Frontend:

```
nodoService.listarNodos();
```

LISTAR CONEXIONES

Frontend:

```
conexionService.listarConexiones();
```

19. MANEJO DE ERRORES

Clase:

```
exceptions/SpatialException.java
```

RESPONSABILIDAD

Errores espaciales.

EJEMPLOS

```
Poste fuera de calle.
```

Geometría inválida.

No existe calle cercana.

20. GITHUB

Rama asignada:

backend-persistence

REGLAS

- No modificar Frontend.
- No modificar Backend 1.
- No modificar Base de Datos.
- No colocar SQL dentro de Services.
- Toda consulta debe estar en Repository.
- Documentar consultas complejas.

21. OBJETIVO FINAL

Al finalizar este módulo deberá ser posible:

- Conectarse a PostgreSQL.
- Utilizar PostGIS.
- Guardar nodos.
- Guardar conexiones.
- Obtener calles.
- Validar postes.
- Construir grafos.
- Comunicar Frontend con Base de Datos.
- Comunicar Base de Datos con Backend 1.
- Gestionar toda la persistencia del sistema.